

Simulador de Gestión de Memoria

Segmentación y Paginación en un Sistema Multiprogramado

Juan Diego Grajales Castillo

20221020128

Steven Navarro Parrales

20221020048

Alejandro Nuñez Barrera

20231020139

18 de mayo de 2026

Resumen

La gestión eficiente de la memoria es un pilar fundamental en los sistemas operativos modernos, especialmente en entornos multiprogramados donde múltiples procesos comparten recursos finitos. El presente informe documenta el desarrollo y análisis de un simulador interactivo basado en JavaScript que modela el comportamiento de la memoria principal bajo dos esquemas avanzados de administración: **Segmentación** y **Paginación**. El simulador considera un espacio de direcciones de 16 MiB (2^{24} direcciones) e implementa algoritmos de asignación como primer ajuste, mejor ajuste y peor ajuste. Los resultados demuestran las ventajas y desventajas de cada método, destacando la fragmentación externa en segmentación y la fragmentación interna en paginación, cumpliendo con el objetivo de evidenciar la importancia de la correcta gestión para evitar inconsistencias en la ejecución de procesos.

1. Introducción

En los sistemas operativos multiprogramados, la memoria RAM es un recurso crítico que debe ser administrado cuidadosamente. Diferentes procesos compiten por el espacio de memoria, y una mala gestión puede llevar a problemas como fragmentación, desperdicio de recursos o incluso fallos en la ejecución. Por ello, se han desarrollado diversos esquemas de gestión: particiones fijas, particiones variables, segmentación y paginación, entre otros. Este trabajo se centra en dos de los más relevantes: **segmentación** (desde la perspectiva del usuario, dividiendo el programa en segmentos lógicos como código, datos y pila) y **paginación** (desde la perspectiva del sistema, dividiendo la memoria física en marcos de tamaño fijo).

1.1. Objetivos

- Reconocer la importancia de la gestión de memoria en un sistema operativo para garantizar la integridad y eficiencia en la ejecución de procesos.
- Implementar un simulador funcional en JavaScript que modele la memoria principal de 16 MiB.
- Aplicar los algoritmos de asignación de memoria (primer ajuste, mejor ajuste, peor ajuste) para los esquemas de segmentación y paginación.
- Analizar el comportamiento de cada método en términos de uso de memoria, fragmentación y capacidad de carga de procesos.

2. Fundamento Teórico

2.1. Gestión de Memoria en Sistemas Multiprogramados

La multiprogramación requiere que el sistema operativo mantenga en memoria varios procesos simultáneamente. Esto implica decidir qué procesos cargar, en qué ubicación y cómo proteger el espacio de unos de otros. Dos estrategias clave son:

- **Segmentación:** divide el programa del usuario en segmentos de tamaño variable (código, datos, pila, heap). Cada segmento se ubica en un área contigua de memoria física, y la tabla de segmentos contiene la base y el límite de cada uno.
- **Paginación:** divide tanto la memoria lógica como la física en bloques de tamaño fijo llamados páginas y marcos, respectivamente. La tabla de páginas traduce direcciones lógicas a físicas.

2.2. Algoritmos de Asignación

Para ubicar los segmentos o páginas en la memoria, se utilizan algoritmos de búsqueda de huecos libres:

- **Primer ajuste (First Fit):** selecciona el primer hueco que sea suficientemente grande.
- **Mejor ajuste (Best Fit):** selecciona el hueco más pequeño que sea suficiente para minimizar el desperdicio.
- **Peor ajuste (Worst Fit):** selecciona el hueco más grande para dejar fragmentos más grandes disponibles.

2.3. Arquitectura de Memoria Simulada

El simulador define:

- Memoria total: 16 MiB = $16 \times 1024 \times 1024 = 16\,777\,216$ bytes.
- Direcciones: 24 bits, rango 0x000000 – 0xFFFFFFFF.
- Memoria reservada para SO: 1 MiB al inicio (direcciones 0x000000 – 0x0FFFFF).
- Memoria de usuario: 15 MiB (direcciones 0x100000 – 0xFFFFFFFF).
- Tamaño de página: 4 KiB = 4096 bytes.
- Número de marcos: $15 \times 1024 \times 1024 / 4096 = 3840$ marcos.

3. Desarrollo del Simulador

El simulador se implementó en JavaScript puro, HTML5 y CSS3, ejecutándose completamente en el navegador. A continuación se describen los componentes principales.

3.1. Estructura de Datos

- **Segmentación:** se mantienen dos listas:

```
1 segmentation = {  
2   segments: [],           // lista de segmentos asignados  
3   freeList: [],           // lista de huecos libres  
4   processMap: {}          // relacion PID -> lista de segmentos  
5 }
```

Listing 1: Estructuras para segmentación

- **Paginación:** se utiliza un arreglo de marcos:

```
1 pagination = {  
2   frames: Array(TOTAL_FRAMES).fill(null), // cada marco guarda {pid,  
3     name, pageIndex}  
4   processMap: {}           // PID -> info del proceso  
5     y sus paginas  
6 }
```

Listing 2: Estructuras para paginación

3.2. Algoritmos de Asignación

Las funciones de búsqueda implementan los tres algoritmos:

```
1 function GetIndexFA(processes, size) // Primer ajuste  
2 function GetIndexBA(processes, size) // Mejor ajuste  
3 function GetIndexWA(processes, size) // Peor ajuste
```

Listing 3: Funciones de asignación

Para segmentación, se trabaja directamente sobre la lista de huecos libres, aplicando el mismo criterio.

3.3. Carga y Liberación de Procesos

Al seleccionar un programa desde el catálogo, el simulador:

1. Calcula el tamaño total requerido (txt + data + bss + heap + stack).
2. Según el método activo, busca espacio suficiente.
3. Si es segmentación, asigna cada segmento individualmente; si hay fragmentación, se rechaza el proceso.
4. Si es paginación, calcula el número de páginas y asigna marcos libres.
5. En caso de éxito, el proceso aparece en la vista de memoria y en la tabla correspondiente.

La liberación de un proceso devuelve sus segmentos o marcos a la lista de libres y fusiona huecos adyacentes (en segmentación) o simplemente marca marcos como nulos (en paginación).

4. Manual de Usuario

El simulador presenta una interfaz dividida en tres áreas principales (ver Figura 1): catálogo de programas (izquierda), panel de gestión (centro) y visualización de memoria (derecha).

4.1. Inicio y selección de método

1. Al cargar la aplicación, se muestra el catálogo de programas predefinidos.
2. Haga clic en el botón “ **Tipo de Gestión**” para desplegar el menú lateral.
3. Seleccione el método deseado: *Segmentación* (con Primer, Mejor o Peor Ajuste) o *Paginación*.
4. El panel central se actualizará mostrando las tablas y estadísticas correspondientes.

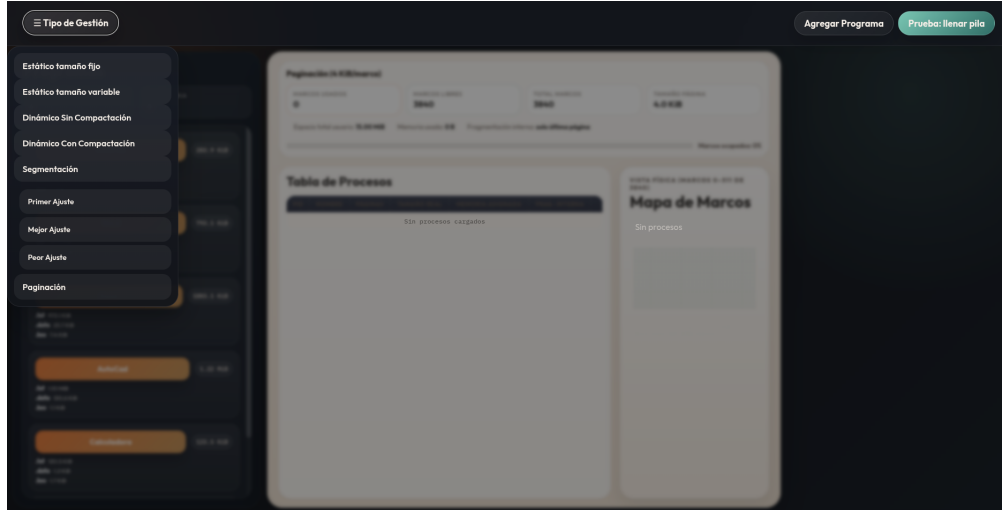


Figura 1: Interfaz principal del simulador con menú de gestión desplegado.

4.2. Carga de programas

1. En el catálogo de la izquierda, cada programa muestra su nombre y tamaño total (ver Figura 2).
2. Haga clic en el botón del programa (ej. *NotePad*, *Word*) para cargarlo en memoria.
3. El simulador aplicará automáticamente el algoritmo de asignación seleccionado.
4. Si el proceso cabe, aparecerá en la tabla de segmentos o en la tabla de procesos (paginación).
5. Si no hay suficiente espacio, se mostrará una alerta y el proceso no se cargará.

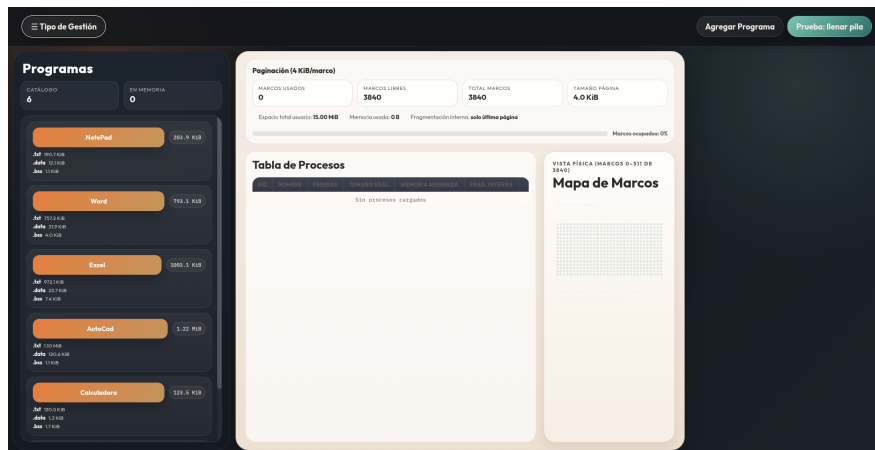


Figura 2: Catálogo de programas con tamaños de texto, datos y BSS.

4.3. Visualización de memoria

- **Segmentación:** se muestra una tabla con todos los segmentos activos (PID, tipo, base, límite, tamaño, permisos) y un mapa de memoria físico coloreado (ver Figura 3). Los huecos libres aparecen en color verde y los segmentos ocupados en tonos cálidos.
- **Paginación:** se presenta una tabla de procesos con el número de páginas, memoria real, memoria asignada y fragmentación interna, junto con una tabla de páginas del último proceso cargado y un mapa de marcos (ver Figura 4).

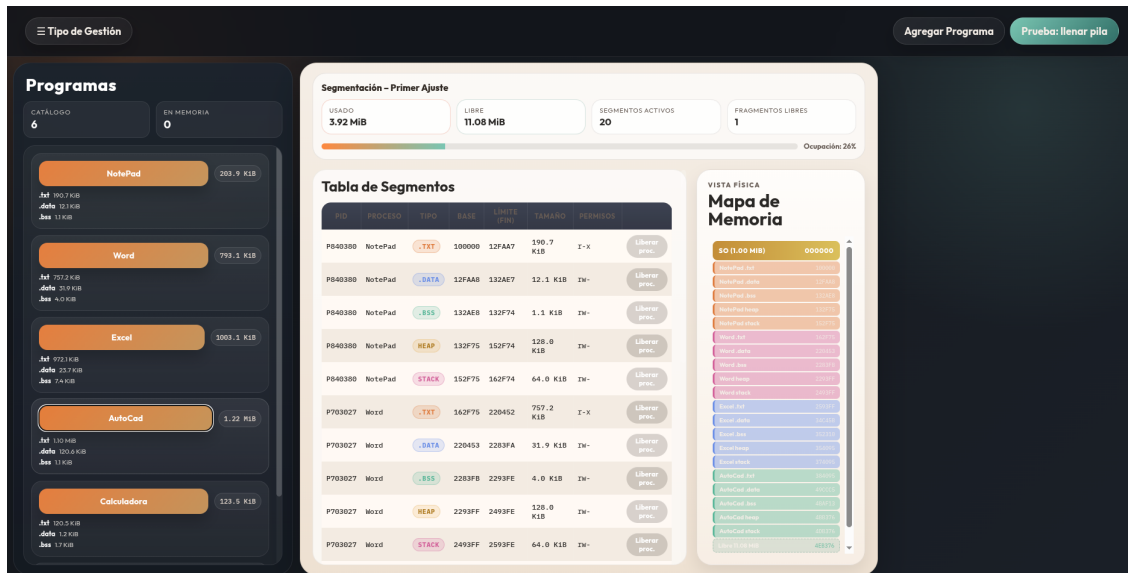


Figura 3: Vista de segmentación mostrando segmentos de NotePad y Word, y mapa de memoria física.

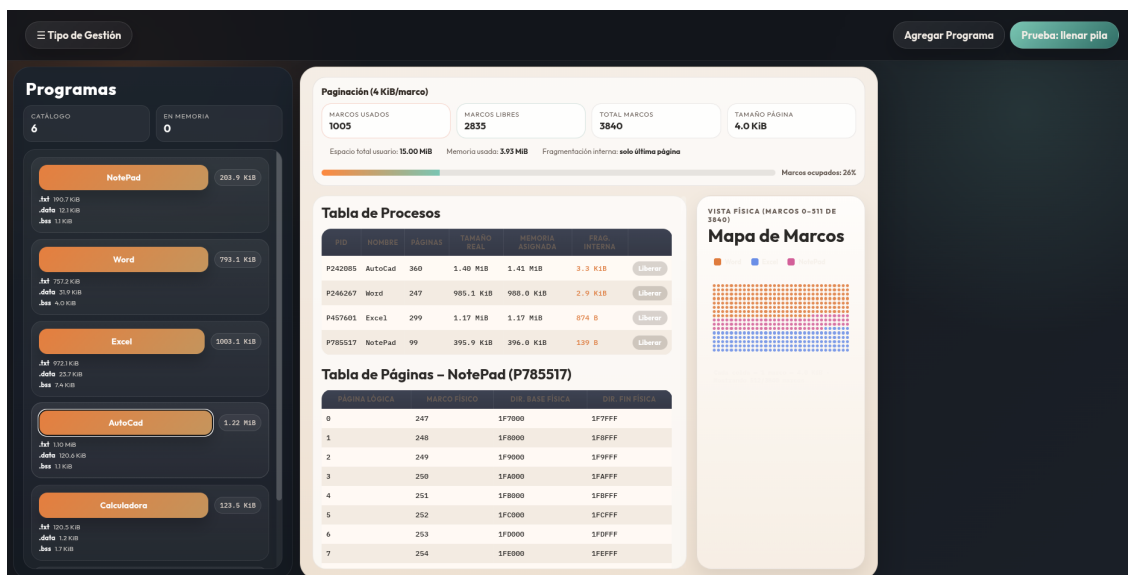


Figura 4: Vista de paginación con tabla de procesos, fragmentación interna y mapa de marcos.

4.4. Liberación de procesos

- En la tabla de segmentos o de procesos, cada fila tiene un botón “**Liberar**”.
- Al hacer clic, el proceso se elimina de la memoria.
- En segmentación, los segmentos se convierten en huecos libres y se fusionan con adyacentes.
- En paginación, los marcos se marcan como libres.

4.5. Añadir nuevos programas

1. Haga clic en “**Agregar Programa**” en la parte superior derecha.
2. Complete el formulario con nombre, tamaño de .txt (bytes), .data (bytes) y .bss (bytes).
3. Envíe el formulario; el nuevo programa aparecerá en el catálogo.

4.6. Botón de prueba “llenar pila”

Este botón carga una serie de procesos de prueba con diferentes tamaños para demostrar el comportamiento de la memoria bajo condiciones de alta ocupación (ver Figura 5).

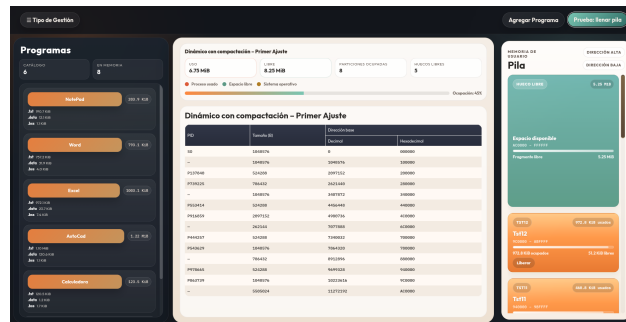


Figura 5: Botón “Prueba: llenar pila” para cargar múltiples procesos de prueba.

5. Pruebas y Resultados

5.1. Configuración de Prueba

Se utilizaron 5 programas predefinidos y un sexto que excede la memoria (para verificar rechazo). Los tamaños se muestran en la Tabla 1.

Cuadro 1: Programas de prueba

Nombre	.txt (B)	.data (B)	.bss (B)	Total (con heap+stack)
NotePad	195.240	12.352	1.165	405.365
Word	775.390	32.680	4.100	1.008.778
Excel	995.420	24.245	7.557	1.223.830
AutoCad	1.150.000	123.470	1.123	1.471.201
Calculadora	123.420	1.246	1.756	323.030
ProgramaErr	5.250.000	3.224.000	51.000	>15 MiB → Excede límite

5.2. Resultados de Segmentación (Primer Ajuste)

Se cargaron los programas en orden: NotePad, Word, Excel, AutoCad, Calculadora. La Tabla 2 muestra los primeros segmentos asignados.

Cuadro 2: Segmentos asignados para NotePad y Word (Primer ajuste)

Proceso	Tipo	Base (hex)	Tamaño (bytes)	Fin (hex)
NotePad	.txt	0x100000	195.240	0x13FA27
NotePad	.data	0x13FA28	12.352	0x142667
NotePad	.bss	0x142668	1.165	0x142B24
NotePad	heap	0x142B25	131.072	0x162524
NotePad	stack	0x162525	65.536	0x172524
Word	.txt	0x172525	775.390	0x230E9E
Word	.data	0x230E9F	32.680	0x238A3A
Word	.bss	0x238A3B	4.100	0x239A3E
Word	heap	0x239A3F	131.072	0x259A3E
Word	stack	0x259A3F	65.536	0x269A3E

Observaciones:

- La segmentación elimina la fragmentación interna, pero genera fragmentación externa.
- Después de cargar los 5 procesos, quedaban huecos libres pequeños (menores a 64 KiB) que impedían cargar un sexto proceso.
- Los algoritmos de ajuste influyen en la posición de los segmentos; primer ajuste tiende a acumular huecos pequeños al inicio.

5.3. Resultados de Paginación

Con la misma carga de procesos, la paginación asignó marcos de 4 KiB. La Tabla 3 muestra el uso de marcos.

Cuadro 3: Uso de marcos para paginación

Proceso	Páginas	Memoria real (bytes)	Memoria asignada	Frag. interna (bytes)
NotePad	99	405.365	396 KiB	139
Word	247	1.008.778	988 KiB	2.934
Excel	299	1.223.830	1.196 KiB	874
AutoCad	360	1.471.201	1.440 KiB	3.359
Calculadora	79	323.030	316 KiB	554

Observaciones:

- La paginación elimina la fragmentación externa pero introduce fragmentación interna (última página incompleta).
- El número total de marcos usados fue $99 + 247 + 299 + 360 + 79 = 1,084$, de 3.840 disponibles (28 % de ocupación).
- La carga de procesos es más flexible porque cualquier marco libre sirve.

5.4. Comparativa de Algoritmos de Ajuste en Segmentación

Se probaron los tres algoritmos con la misma secuencia de carga:

- **Primer ajuste:** más rápido, pero tiende a fragmentar la memoria tempranamente.
- **Mejor ajuste:** deja huecos muy pequeños que se vuelven inútiles.
- **Peor ajuste:** deja huecos más grandes y balanceados, permitiendo cargar procesos adicionales en algunos casos.

6. Conclusiones

El simulador desarrollado cumple con los objetivos planteados:

1. Demuestra la importancia de la gestión de memoria en un sistema multiprogramado, mostrando cómo diferentes estrategias afectan la capacidad de cargar procesos y la eficiencia en el uso de memoria.
2. Implementa correctamente los métodos de segmentación y paginación, así como los algoritmos de asignación (primer, mejor, peor ajuste).
3. La segmentación ofrece una visión lógica cercana al programador, pero sufre de fragmentación externa. La paginación simplifica la gestión a costa de fragmentación interna.
4. El simulador es interactivo y visual, facilitando la comprensión de conceptos abstractos.

7. Referencias

- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Sistema Operativos*. 9^a ed. Wiley.
- Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems*. 4^a ed. Pearson.
- Stallings, W. (2017). *Operating Systems: Internals and Design Principles*. 9^a ed. Pearson.